Writing your trader.py file

This guide assumes familiarity with Python and Object-Oriented Programming.

The algorithmic trading file is the same format as the file used in IMC Prosperity. If you've participated in that competition, you may only find the backtesting logic section relevant.

General Information

For QuantiFi, you will be writing and uploading a trading algorithm class written in Python, which will be tested on our backtesting engine. On this engine, the algorithm will trade against multiple bots.

The format for the trading algorithm will be a predefined Trader class, which must contain a run() method that contains all trading logic. This function must have parameters self and a TradingState object. You can read the TradingState tab on this Doc for more information.

To test your algorithm, you can upload it onto https://www.utefa.com/quantifi after logging in with the credentials given by the exec team. Once uploaded, your algorithm will run in the simulation environment and backtest against historical data. During each tick, the run method will be called and provided with a TradingState object. This object contains an overview of all trades that have happened since the last iteration. The TradingState will contain a per product overview of all the outstanding buy and sell orders (also called "quotes").

Based on the logic in the run() function, your algorithm can decide to send orders. If you send a buy order at a price equal or higher than one of the bot sell orders, it will result in a trade. If your quantity for that trade is larger than the bot's quote, then the remaining quantity will be left as an outstanding quote.

Every trade done by the algorithm changes your position. You can hold a negative position. Suppose you have 5 shares in NVIDIA and you sell 10. Your position is now -5, also referred to as being (short 5). The algorithm is restricted by per product limits defined later in this section. They are the absolute position (long or short) that the

algorithm must stay within. If your run() sends a quote exceeding this, your order will be automatically cancelled.

Trader class

Your Python file **must** contain a Trader class with a run() function. Below is an example buying as many shares of NVIDIA as possible, while under a position limit of 100.

```
from datamodel import TradingState, Order
from typing import List

class Trader:

    def run(self, state: TradingState):
        result = {}
        orders: List[Order] = []

        # check our current position
        position = state.position.get("NVIDIA", 0)
        order_depth = state.order_depths["NVIDIA"]

        # assuming position limit is 100
        if position < 100 and order_depth.sell_orders:
            # get best sell (ask) price
            best_ask = min(order_depth.sell_orders.keys())

            # get the number of shares being sold at the best selling price
            # take the negative, since sell quantities are stored as negative numbers
            available = -order_depth.sell_orders[best_ask]

            # determine the maximum quantity we can buy
            quantity = min(100 - position, available)

            # append our order to the results object
            orders.append(Order(product, best_ask, quantity))

        # map the ticker to the orders we're submitting
        result["NVIDIA"] = orders
```

```
        # expected to return results, conversions, and traderData
        return result, 0, ""
```

You can find an additional example that trades the AMETHYST product in the Google Drive folder containing the data.

Return Type

The returns are result, conversions, and traderData.

results is a dictionary containing all the orders your trader wants to place.

```
result = {
      "NVIDIA": [Order("NVIDIA", 150, 20)], # buy 20 NVIDIA at 150$
      "AAPL": [Order("AAPL", 200, -30)] # sell 30 AAPL at 200$
}
```

conversions is a mechanism in IMC Prosperity that we will not be using. You can leave it at 0.

traderData is the information you want to remember. You can't store information in the TradingState object, so you will store it here. For example, you can store the past 100 prices of a stock. traderData must be a string, so we recommend you convert your data into a json before returning.

```
import json

class Trader:
    def run(self, state: TradingState):

        # Load previous data, or start with an empty list
        if state.traderData:
            prices = json.loads(state.traderData)
        else:
            prices = []

        # Add a new price
```

```
        prices.append(150)

        # Convert the list to a string
        traderData = json.dumps(prices)

        return {}, 0, traderData
```

Data

You are given two .csv files. Prices and Trades. They are on days -2, -1, and 0. Day 1 will be the OOS scoring day. Each day has 10,000 ticks. The timestamps are from 0 to 999,900 in steps of 100. The data is provided in semicolon-separated csv files.

Prices

Prices contain up to three bid and ask levels per product.

Column Name	Meaning
day	Trading day: -2, -1, 0 in the training data
timestamp	Ticks within trading day, from 0 to 999,900 in steps of 100
product	Product name
bid_price_1, bid_price_2, bid_price_3	Buy prices, ordered highest to lowest. Level 1 is the best bid.
bid_volume_1, bid_volume_2, bid_volume_3	Quantity available at each bid price.
ask_price_1, ask_price_2, ask_price_3	Sell prices, ordered lowest to highest. Level 1 is the best ask.
ask_volume_1, ask_volume_2, ask_volume_3	Quantity available at each ask price
mid_price	Arithmetic average of the best bid and the best ask.

profit_and_loss	Placeholder. value is always 0

Trades

Trades show the executed market trades. Counterparty information is empty.

Column Name	Meaning
timestamp	Time of the trade within the day
buyer	Counterparty identity. Always blank.
seller	Counterparty identity. Always blank.
symbol	Product traded. It's the same as the product column in prices.csv but it has a different name.
currency	Currency value is always "COINS"
price	Transaction price
quantity	Number of units traded

Backtesting

In the QuantiFi backtester, you will be able to see the full backtest on historical data (days -2, -1, and 0).

The backtester gives you a passive fill if your price was equal or better than the price someone actually traded at.

For example: you place a buy order at 100. Consider the following scenarios.

- A sell happens at 99 -> your order gets executed at 100, because the seller would prefer a higher selling price at 100.
- A sell happens at 100 -> your order gets executed, an equal-price historical trade counts as a fill

- A sell happens at 101 -> your order doesn't get executed, because the seller found a better price at 101.

Note: you can assume that the order book in TradingState and the .csv files are never empty.